

SIMATS ENGINEERING

Department of Computer Science & Engineering

CO4-AT1 - Industrial Problem Solving | Programming in Java

Student Name	K. V. S. S. Jeswanth Babu
Register No.	192472012
Course Code	CSA0904
Course Name	Programming in Java
Assignment	CO4-AT1 - Industrial Problem Solving (Question 2)

QUESTION 2: GUI-BASED BANKING & FUND TRANSFER APPLICATION

Problem Statement:

Develop a GUI-based banking application that allows users to perform operations such as fund transfer, balance inquiry, and viewing transaction history. Use Swing components and event listeners to manage user interactions securely. Implement input validation for sensitive data like account number and amount. Ensure proper error handling for invalid transactions. Apply suitable layout managers to create a responsive and user-friendly interface. Analyze system responsiveness during multiple operations and suggest improvements. Demonstrate how secure input handling enhances reliability.

SOURCE CODE

```
import java.awt.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.List;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

// Main Public Class at the Top
public class BankAccountSystem extends JFrame {
    // In-memory Database of Accounts
    private static final Map<String, BankAccount> accountsDB = new HashMap<>();

    static {
        // Seed test accounts
        accountsDB.put("1001", new BankAccount("1001", "Jeswanth Babu", "1234", 25000.00));
        accountsDB.put("1002", new BankAccount("1002", "Alex Morgan", "5678", 12000.00));
        accountsDB.put("1003", new BankAccount("1003", "Robert Downey", "4321", 50000.00));
    }

    private final JTextField txtSourceAcc;
    private final JPasswordField txtPin;
    private final JTextField txtDestAcc;
    private final JTextField txtAmount;
    private final JLabel lblDisplayBalance;
    private final JLabel lblDisplayHolder;
    private final DefaultTableModel tableModel;
    private final JTable historyTable;

    public BankAccountSystem() {
        setTitle("Enterprise GUI Banking & Fund Transfer Portal");
    }
}
```

```

setSize(850, 650);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setLocationRelativeTo(null);
setLayout(new BorderLayout(10, 10));

// Header Panel
JPanel pnlHeader = new JPanel();
pnlHeader.setBackground(new Color(21, 67, 96));
JLabel lblHeader = new JLabel("SECURE DIGITAL BANKING SYSTEM");
lblHeader.setForeground(Color.WHITE);
lblHeader.setFont(new Font("Arial", Font.BOLD, 18));
pnlHeader.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
pnlHeader.add(lblHeader);
add(pnlHeader, BorderLayout.NORTH);

// Center Panel: Split into Operations Form and Transaction History
JPanel pnlCenter = new JPanel(new GridLayout(1, 2, 15, 15));
pnlCenter.setBorder(BorderFactory.createEmptyBorder(10, 15, 10, 15));

// Left Sub-Panel: Banking Actions
JPanel pnlOperations = new JPanel(new GridBagLayout());
pnlOperations.setBorder(BorderFactory.createTitledBorder("Authentication & Operations"));
GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(6, 6, 6, 6);
gbc.fill = GridBagConstraints.HORIZONTAL;

// Row 0: Source Account
gbc.gridx = 0; gbc.gridy = 0;
pnlOperations.add(new JLabel("Your Account No:"), gbc);
gbc.gridx = 1;
txtSourceAcc = new JTextField(10);
pnlOperations.add(txtSourceAcc, gbc);

// Row 1: Security PIN
gbc.gridx = 0; gbc.gridy = 1;
pnlOperations.add(new JLabel("Security PIN:"), gbc);
gbc.gridx = 1;
txtPin = new JPasswordField(10);
pnlOperations.add(txtPin, gbc);

// Row 2: Inquire Balance Button
gbc.gridx = 0; gbc.gridy = 2; gbc.gridwidth = 2;
JButton btnCheckBalance = new JButton("Inquire Balance / Load Account");
btnCheckBalance.setBackground(new Color(41, 128, 185));
btnCheckBalance.setForeground(Color.WHITE);
btnCheckBalance.addActionListener(e -> handleBalanceInquiry());
pnlOperations.add(btnCheckBalance, gbc);

// Row 3: Account Info Display Box
gbc.gridx = 0; gbc.gridy = 3; gbc.gridwidth = 2;
JPanel pnlInfoCard = new JPanel(new GridLayout(2, 1));
pnlInfoCard.setBackground(new Color(235, 245, 251));
pnlInfoCard.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createLineBorder(new Color(174, 214, 241)),
    BorderFactory.createEmptyBorder(8, 8, 8, 8)
));
lblDisplayHolder = new JLabel("Holder Name: --");
lblDisplayHolder.setFont(new Font("SansSerif", Font.PLAIN, 12));
lblDisplayBalance = new JLabel("Available Balance: ₹0.00");
lblDisplayBalance.setFont(new Font("SansSerif", Font.BOLD, 13));
lblDisplayBalance.setForeground(new Color(39, 174, 96));
pnlInfoCard.add(lblDisplayHolder);
pnlInfoCard.add(lblDisplayBalance);
pnlOperations.add(pnlInfoCard, gbc);

// Row 4: Separator

```

```

gbc.gridx = 0; gbc.gridy = 4; gbc.gridwidth = 2;
pnlOperations.add(new JSeparator(), gbc);

// Row 5: Destination Account
gbc.gridx = 0; gbc.gridy = 5; gbc.gridwidth = 1;
pnlOperations.add(new JLabel("Target Account No:"), gbc);
gbc.gridx = 1;
txtDestAcc = new JTextField(10);
pnlOperations.add(txtDestAcc, gbc);

// Row 6: Transfer Amount
gbc.gridx = 0; gbc.gridy = 6;
pnlOperations.add(new JLabel("Amount (₹):"), gbc);
gbc.gridx = 1;
txtAmount = new JTextField(10);
pnlOperations.add(txtAmount, gbc);

// Row 7: Transfer Button
gbc.gridx = 0; gbc.gridy = 7; gbc.gridwidth = 2;
JButton btnTransfer = new JButton("Execute Fund Transfer");
btnTransfer.setBackground(new Color(39, 174, 96));
btnTransfer.setForeground(Color.WHITE);
btnTransfer.setFont(new Font("SansSerif", Font.BOLD, 12));
btnTransfer.addActionListener(e -> handleFundTransfer());
pnlOperations.add(btnTransfer, gbc);

pnlCenter.add(pnlOperations);

// Right Sub-Panel: Transaction History Table
JPanel pnlHistory = new JPanel(new BorderLayout(5, 5));
pnlHistory.setBorder(BorderFactory.createTitledBorder("Live Transaction Audit Trail"));

String[] columns = {"Timestamp", "Type", "Amount", "Balance", "Details"};
tableModel = new DefaultTableModel(columns, 0) {
    @Override
    public boolean isCellEditable(int row, int column) { return false; }
};
historyTable = new JTable(tableModel);
historyTable.setFont(new Font("SansSerif", Font.PLAIN, 11));
historyTable.getTableHeader().setFont(new Font("SansSerif", Font.BOLD, 11));
historyTable.setRowHeight(22);
JScrollPane scrollPane = new JScrollPane(historyTable);
pnlHistory.add(scrollPane, BorderLayout.CENTER);

pnlCenter.add(pnlHistory);
add(pnlCenter, BorderLayout.CENTER);

// Status Footer
JPanel pnlFooter = new JPanel(new FlowLayout(FlowLayout.CENTER));
JLabel lblStatus = new JLabel("• TLS / AES Emulated Session • Real-Time Input Validation Active • Test Account  
ts: 1001 (PIN 1234), 1002 (PIN 5678), 1003 (PIN 4321)");
lblStatus.setFont(new Font("SansSerif", Font.ITALIC, 11));
lblStatus.setForeground(Color.GRAY);
pnlFooter.add(lblStatus);
add(pnlFooter, BorderLayout.SOUTH);
}

// Authenticate and fetch account
private BankAccount authenticateUser(String accNum, char[] pin) throws BankingTransactionException {
    if (accNum.isEmpty()) {
        throw new BankingTransactionException("Account number field cannot be empty.");
    }
    if (!accNum.matches("^\\d{4,10}$")) {
        throw new BankingTransactionException("Account number must be numeric (4-10 digits).");
    }
    if (pin == null || pin.length == 0) {

```

```

        throw new BankingTransactionException("PIN cannot be empty.");
    }

    BankAccount account = accountsDB.get(accNum);
    if (account == null) {
        throw new BankingTransactionException("Account number '" + accNum + "' not found.");
    }
    if (!account.validatePin(pin)) {
        throw new BankingTransactionException("Authentication failed: Invalid PIN provided.");
    }
    return account;
}

// Handler for Balance Inquiry
private void handleBalanceInquiry() {
    try {
        String accNum = txtSourceAcc.getText().trim();
        char[] pin = txtPin.getPassword();

        BankAccount account = authenticateUser(accNum, pin);

        // Update Info Card
        lblDisplayHolder.setText("Holder Name: " + account.getHolderName());
        lblDisplayBalance.setText("Available Balance: ₹" + String.format("%.2f", account.getBalance()));

        // Refresh History Table
        refreshTable(account);

        JOptionPane.showMessageDialog(this,
            "Account Loaded Successfully!\nHolder: " + account.getHolderName() +
            "\nCurrent Balance: ₹" + String.format("%.2f", account.getBalance()),
            "Account Summary", JOptionPane.INFORMATION_MESSAGE);

    } catch (BankingTransactionException ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage(), "Authentication / Inquiry Error", JOptionPane.ERROR_
MESSAGE);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Unexpected Error: " + ex.getMessage(), "System Fault", JOptionPane.E
RROR_MESSAGE);
    }
}

// Handler for Fund Transfer
private void handleFundTransfer() {
    try {
        String srcAccNum = txtSourceAcc.getText().trim();
        char[] pin = txtPin.getPassword();
        String destAccNum = txtDestAcc.getText().trim();
        String amountStr = txtAmount.getText().trim();

        // 1. Authenticate Sender
        BankAccount srcAccount = authenticateUser(srcAccNum, pin);

        // 2. Validate Destination Account
        if (destAccNum.isEmpty()) {
            throw new BankingTransactionException("Destination account number cannot be empty.");
        }
        if (!destAccNum.matches("^\\d{4,10}$")) {
            throw new BankingTransactionException("Destination account must be numeric (4-10 digits).");
        }
        BankAccount destAccount = accountsDB.get(destAccNum);
        if (destAccount == null) {
            throw new BankingTransactionException("Destination account '" + destAccNum + "' does not exist.");
        }

        // 3. Validate Amount
    }
}

```

```

        if (amountStr.isEmpty()) {
            throw new BankingTransactionException("Transfer amount cannot be empty.");
        }
        double amount;
        try {
            amount = Double.parseDouble(amountStr);
        } catch (NumberFormatException nfe) {
            throw new BankingTransactionException("Invalid amount format. Please enter a valid decimal number.");
        }

        // 4. Confirmation Dialog
        int confirm = JOptionPane.showConfirmDialog(this,
            "Confirm Transfer of ₹" + String.format("%.2f", amount) + " to " +
            destAccount.getHolderName() + " (A/C: " + destAccNum + ")?",
            "Authorize Fund Transfer", JOptionPane.YES_NO_OPTION);

        if (confirm != JOptionPane.YES_OPTION) {
            return;
        }

        // 5. Execute Atomic Transfer
        srcAccount.transfer(destAccount, amount);

        // 6. Update UI
        lblDisplayBalance.setText("Available Balance: ₹" + String.format("%.2f", srcAccount.getBalance()));
        txtAmount.setText("");
        txtDestAcc.setText("");
        refreshTable(srcAccount);

        JOptionPane.showMessageDialog(this,
            "Transfer Successful!\n₹" + String.format("%.2f", amount) +
            " sent to " + destAccount.getHolderName() + ".\nRemaining Balance: ₹" +
            String.format("%.2f", srcAccount.getBalance()),
            "Transaction Completed", JOptionPane.INFORMATION_MESSAGE);

    } catch (BankingTransactionException ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage(), "Transaction Failed", JOptionPane.WARNING_MESSAGE);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "An error occurred: " + ex.getMessage(), "Transaction Error", JOptionPane.ERROR_MESSAGE);
    }
}

private void refreshTable(BankAccount account) {
    tableModel.setRowCount(0);
    for (String[] row : account.getTransactionHistory()) {
        tableModel.addRow(row);
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new BankAccountSystem().setVisible(true));
}

// Custom Exception for Banking Business Rules
class BankingTransactionException extends Exception {
    public BankingTransactionException(String message) {
        super(message);
    }
}

// Model Class representing an Account
class BankAccount {
    private final String accountNumber;
    private final String holderName;

```

```

private final String pin;
private double balance;
private final List<String[]> transactionHistory;

public BankAccount(String accountNumber, String holderName, String pin, double initialBalance) {
    this.accountNumber = accountNumber;
    this.holderName = holderName;
    this.pin = pin;
    this.balance = initialBalance;
    this.transactionHistory = new ArrayList<>();
    recordTransaction("ACCOUNT_OPENED", initialBalance, "Initial Balance Deposit");
}

public String getAccountNumber() { return accountNumber; }
public String getHolderName() { return holderName; }
public double getBalance() { return balance; }

public boolean validatePin(char[] inputPin) {
    if (inputPin == null) return false;
    String pinStr = new String(inputPin);
    boolean isValid = this.pin.equals(pinStr);
    Arrays.fill(inputPin, '0');
    return isValid;
}

public synchronized void transfer(BankAccount targetAccount, double amount) throws BankingTransactionException {
    if (amount <= 0) {
        throw new BankingTransactionException("Transfer amount must be strictly greater than 0.");
    }
    if (this.balance < amount) {
        throw new BankingTransactionException("Insufficient funds! Available balance: ₹" + String.format("%.2f",
this.balance));
    }
    if (this.accountNumber.equals(targetAccount.getAccountNumber())) {
        throw new BankingTransactionException("Source and Destination accounts cannot be identical.");
    }

    this.balance -= amount;
    targetAccount.balance += amount;

    this.recordTransaction("TRANSFER_DEBIT", amount, "Sent to A/C: " + targetAccount.getAccountNumber());
    targetAccount.recordTransaction("TRANSFER_CREDIT", amount, "Received from A/C: " + this.accountNumber);
}

private synchronized void recordTransaction(String type, double amount, String description) {
    String timestamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));
    transactionHistory.add(new String[]{
        timestamp,
        type,
        String.format("₹%.2f", amount),
        String.format("₹%.2f", balance),
        description
    });
}

public List<String[]> getTransactionHistory() {
    return Collections.unmodifiableList(transactionHistory);
}
}

```

OUTPUT SCREENSHOT

